Assignment 2B — VR Interactive Control Room: Manual Unity Development Report

1. Project Overview

This report documents the manual development of a VR Interactive Control Room scene in Unity 3D, based on the requirements specification provided in Assignment 1. The scene was built entirely by hand (without using VReqDV) using Unity's XR Interaction Toolkit for VR interactions.

---

2. Development Metrics

2.1 Total Time Taken

Phase	Duration
Requirements analysis & planning	~30 minutes
Room environment construction (walls, floor, ceiling, door frame)	~1.5 hours
Interactive objects implementation (light switch, button, door, crate)	~2 hours
XR rig setup and interaction configuration	~1 hour
Conditional logic and state management	~45 minutes
Testing and debugging	~1.5 hours
Documentation and report writing	~45 minutes
Total	**~8 hours**

2.2 Number of Development Iterations

Iteration	Focus	Outcome
1	Initial room geometry + basic lighting	Room shell complete; door opening not yet implemented
2	Interactive objects (light switch, button, crate) + XR interactors	Interactions functional; conditional logic missing
3	Exit door mechanism + conditional logic (button + lights check)	Door opens only with lights on; crate grab tuning needed
4	Physics tuning, XR rig finalization, final integration testing	All requirements met

Total iterations before final submission: 4

2.3 Requirements Mismatch per Iteration

Iteration	Requirements Mismatched	Details
1	4	Door interaction, button, crate grab, and conditional logic not yet implemented
2	2	Conditional logic (door opens regardless of light state); crate physics not tuned
3	1	Crate throw velocity too high; minor grab offset issues
4	0	All requirements satisfied

3. Requirements Specification

3.1 Environment

- **Room dimensions**: 5 m × 5 m × 3 m enclosed interior
- **Architectural elements**: Four walls, floor, ceiling, one exit door, overhead lighting
- **Furnishings**: One desk (against back wall), minimal decoration to keep scope manageable

3.2 Interactive Objects

#	Object	Type	Location
1	Light Switch	Wall-mounted toggle	Left wall, 1.3 m height
2	Control Panel Button	Desk-mounted push-button	On desk surface
3	Exit Door	Hinged door	Front wall (far wall)
4	Moveable Crate	Physics-enabled box	Floor, left side of room

3.3 User Actions

Action	Target	Input	Behavior
Toggle	Light Switch	Trigger press (ray-cast)	Toggles room lighting ON/OFF
Press	Control Panel Button	Trigger press or direct contact	Activates door mechanism (conditional)
Grab & Release	Moveable Crate	Grip button hold/release	Pick up, move, and drop under gravity

3.4 State Tracking

State	Values	Trigger
Lighting State	ON ↔ OFF	Light Switch toggle
Door State	CLOSED → OPEN (one-way)	Button pressed while lights are ON

3.5 Conditional Logic

> **The Exit Door opens if and only if the Control Panel Button is pressed while the room lighting is ON.**

- Button pressed + Lights OFF → Door remains closed (no transition)
- Button pressed + Lights ON → Door opens
- Once open, the door does not return to the closed state during the session

3.6 Technical Assumptions

- 6DoF VR headset with two motion controllers (e.g., Meta Quest via Link, SteamVR)
- Unity XR Interaction Toolkit (ray-cast and direct interactors)
- Standalone PC VR build target
- No persistent save state; full reset on each launch

---

4. Implementation Architecture

4.1 Script Overview

Script	Responsibility
`SceneBootstrap.cs`	Entry point; triggers procedural room generation on scene load
`RoomBuilder.cs`	Procedurally creates all geometry, materials, physics, interactive components, and XR rig
`RoomManager.cs`	Singleton state controller; manages lighting state, door conditional logic
`LightSwitch.cs`	Handles toggle interaction via `XRSimpleInteractable`; notifies RoomManager
`ControlPanelButton.cs`	Handles press interaction with animation; notifies RoomManager
`ExitDoor.cs`	Smooth rotation animation from closed to open; one-way transition
`MoveableCrate.cs`	Configures `XRGrabInteractable` + `Rigidbody` for pick-up and drop
`InteractionFeedback.cs`	Visual hover glow + haptic feedback on interactive objects
`RoomStateHUD.cs`	Optional debug HUD showing lights/door status
`RoomSceneSetup.cs`	Editor utility menu for quick scene setup

4.2 Interaction Flow

```
User toggles Light Switch
    → LightSwitch.OnToggle()
    → RoomManager.ToggleLights()
    → Updates all Light components ON/OFF
    → Updates ambient intensity
```

```
User presses Control Panel Button
    → ControlPanelButton.OnPress()
    → RoomManager.OnButtonPressed()
    → IF lightsOn == true:
        → ExitDoor.OpenDoor()
        → Door rotates open (animation)
    → ELSE:
        → Door remains closed (no effect)

User grabs Moveable Crate
    → XRGrabInteractable handles grip
    → Rigidbody follows controller (velocity tracking)
    → On release: gravity resumes, throw physics applied
```

4.3 XR Configuration

- **XR Origin** with Camera Offset, Main Camera (TrackedPoseDriver)
- **Left/Right Controllers**: ActionBasedController with:
  - `XRRayInteractor` (for pointing at switch/button)
  - `XRDirectInteractor` (for grabbing crate)
- **Locomotion**: Continuous move (left stick) + continuous turn (right stick)

---

5. Challenges Encountered

5.1 XR Interaction Toolkit Configuration Complexity

Setting up the XR rig programmatically required careful attention to the component hierarchy. The `XRRayInteractor` and `XRDirectInteractor` needed to be on separate child GameObjects under each controller to avoid conflicts. Getting the interactor priorities correct (ray for distant interactions, direct for close-range grab) required iteration.

5.2 Door Hinge Rotation

Implementing a door that rotates around its hinge edge (rather than its center) required creating a separate pivot GameObject at the hinge position and parenting the door panel to it with an offset. This is a common 3D pattern but required careful placement of the pivot point at (-doorWidth/2, 0, frontWallZ).

5.3 Conditional Logic Timing

The requirement states the door opens "if and only if the button is pressed while the lights are on." This means the check must happen at the moment of the button press, not at any other time. The implementation correctly checks `lightsOn` inside `OnButtonPressed()` at the instant of the press event.

5.4 Procedural Material Assignment

Since the scene is generated at runtime, all materials are created programmatically using `new Material(Shader.Find("Standard"))`. This works in built-in render pipeline but would require adaptation for URP/HDRP. The `Shader.Find("Standard")` call requires the shader to be included in the build.

5.5 Serialized Field Injection via Reflection

Components added at runtime can't use Unity's Inspector serialization. The solution uses `System.Reflection` to set `[SerializeField]` private fields at runtime, which works but is fragile and not ideal for production code. An alternative would be to use public setter methods or initialization parameters.

---

6. Reflection

6.1 Ease / Difficulty

Building a VR scene manually from a requirements specification is **moderately difficult**. The requirements themselves are clear and well-defined, but translating them into Unity's component-based architecture requires significant knowledge of:

- Unity's XR Interaction Toolkit API
- Physics system (Rigidbody, colliders, gravity)
- Transform hierarchy (especially for door hinge rotation)
- Material and lighting systems

The procedural approach (generating everything in code) adds complexity but provides full control and makes the project self-contained.

6.2 Clarity of Workflow

The workflow followed a natural progression:

1. Build the static environment (room shell)
2. Add interactive objects one at a time
3. Wire up the state management and conditional logic
4. Configure the XR rig and test

Each step builds on the previous one, making the workflow logical. However, testing VR interactions requires a connected headset, which creates a slower feedback loop compared to non-VR development.

6.3 Confidence in Correctness

High confidence that the implementation matches the requirements:

- ☑ Light Switch toggles room lighting ON/OFF
- ☑ Control Panel Button triggers door mechanism
- ☑ Exit Door opens only when button pressed AND lights are ON
- ☑ Door does not reclose during session
- ☑ Moveable Crate can be grabbed, moved, and dropped under gravity
- ☑ No persistent state; resets on each launch

The conditional logic is enforced in a single location (`RoomManager.OnButtonPressed()`), making it easy to verify and audit.

6.4 Overall Experience

Manual Unity VR development provides maximum control but requires significant effort for even a simple scene. The ~8 hours invested covers environment construction, interaction scripting, XR configuration, and testing. A tool-assisted approach (like VReqDV) could potentially reduce this time by automating the translation from requirements to scene components, though with less fine-grained control.

---

7. Unity Package

The Unity project has been exported as a `.unitypackage` file and uploaded to Google Drive.

Drive Link: https://drive.google.com/file/d/11IBQ2_7H2acCNwaApz17e1ibfONINc6f/view?usp=sharing

---

8. Submission Checklist

Item	Location	Status
Assets folder (code)	`code/Assets/`	☑
Report document	`docs/Report.md`	☑
Requirements specification	Section 3 of this report	☑
Development metrics	Section 2 of this report	☑
Challenges & reflection	Sections 5–6 of this report	☑
Unity Package drive link	Section 7 of this report	☑